NOTE You can read about the service locator antipattern in *Dependency Injection Principles, Practices, and Patterns*, by Steven van Deursen and Mark Seemann (Manning, 2019).

In this chapter we covered some of the reasons to use DI in your applications, how to enable optional ASP.NET Core features by adding services to the DI container, and how to access services from the DI container by using injection into your endpoint handlers. In chapter 9 you'll learn about service lifetimes and how to register your own services with the DI container.

Summary

- DI is baked into the ASP.NET Core framework. You need to ensure that your application adds all the framework's dependencies for optional features in Program.cs; otherwise, you'll get exceptions at runtime when the DI container can't find the required services.
- The dependency graph is the set of objects that must be created to create a specific requested root object. The DI container creates all these dependencies for you.
- You should aim to use explicit dependencies instead of implicit dependencies in most cases. ASP.NET Core uses constructor arguments and endpoint handler arguments to declare explicit dependencies.
- When discussing DI, the term *service* is used to describe any class or interface registered with the container.